



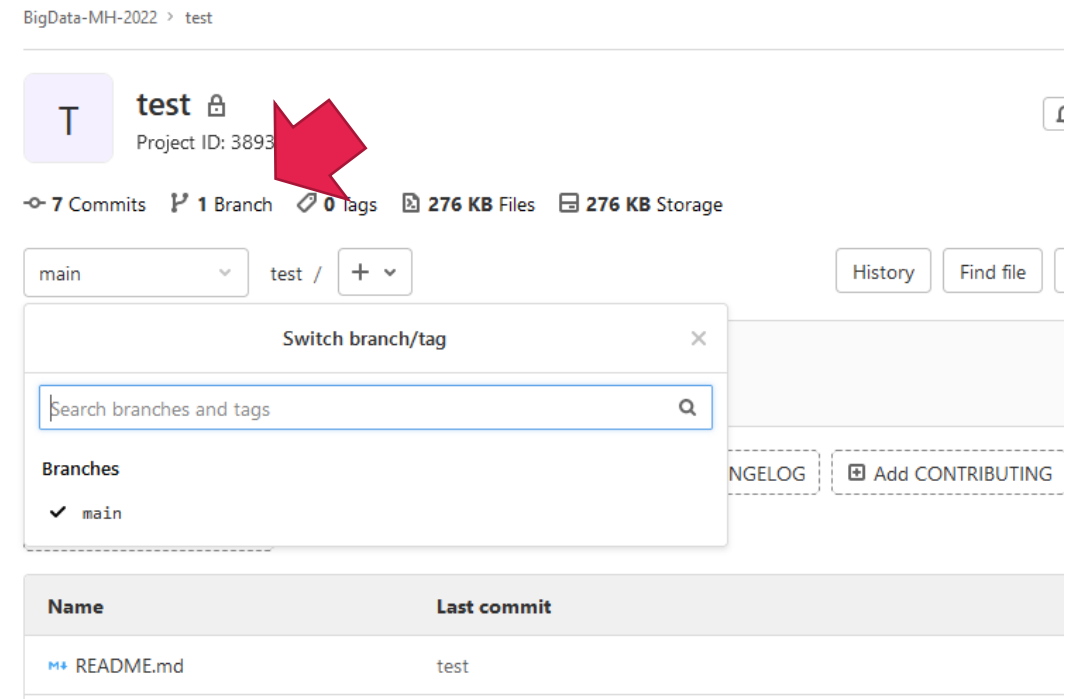
GIT BRANCHES

Big Data H/M 2022

Richard McCreadie

GOOD SOFTWARE ENGINEERING PRACTICE WITH GIT

- So far we have discussed the basics of working with git for revision control
 - However, having everyone commit to the main branch of a git project is problematic for managing parallel development, so we need to discuss [personal branches](#)
- When we created the new git project, what it displayed to us was the [main branch](#)



WHAT IS A BRANCH

- A branch represents a sequence of changes to your project, the main branch is simply the original one of these

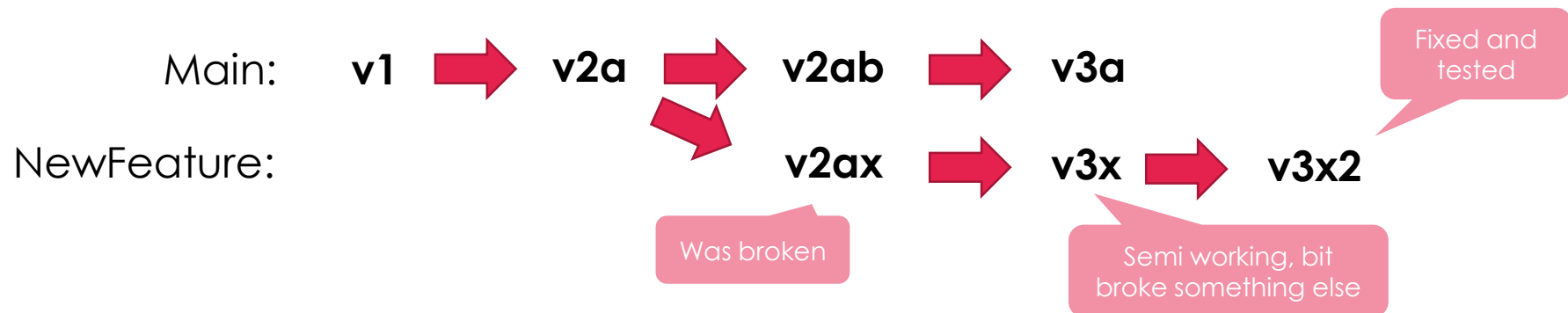
Main: **v1** → **v2a** → **v2ab**

- At any commit, you can create a new branch, which is sort of like an alternative future of the code-base
 - Committed code only affect the branch it is committed to

Main: **v1** → **v2a** → **v2ab** → **v3a**
MyBranch: ↘ **v2ax** → **v3x**

WHY BRANCH?

- Why would you want to have multiple parallel copies of a project?
 - Typically, if you are working on a feature that is not yet complete, you might create a branch for that feature, in this way you can continue to work on and test that feature without breaking the main branch for other people



WHY NOT TO BRANCH?

- The downside of this is that you then need to merge the changes back into the main branch afterward, which may be complicated depending on how many other changes were made to the main branch since your branch was created

